MACHINE LEARNING LABORATORY

Experiment No. 05

Polynomial Regression on California Housing Dataset Using Python

Aim

To implement Polynomial Regression using Python on the California Housing Dataset and study the difference between Linear Regression and Polynomial Regression for predicting housing prices.

Objectives

1. To understand the concept of regression analysis.
2. To study Simple Linear Regression, Multiple Linear Regression, and Polynomial Regression.
3. To understand non-linear relationships between variables.
4. To implement Polynomial Regression using Scikit-Learn.
5. To predict housing prices using machine learning techniques.
6. To evaluate the performance of the regression model using appropriate metrics.

Theory

Regression Analysis

Regression is a supervised machine learning technique used to predict a continuous numerical value based on one or more independent variables. It helps establish the relationship between input features and a target variable.

Applications of regression include:

- House price prediction
- Stock market analysis
- Weather forecasting
- Sales prediction
- Demand forecasting

Simple Linear Regression

Simple Linear Regression is used when there is only one independent variable and one dependent variable. It assumes a linear relationship between the variables.

Mathematical Model

Where:

- = Dependent Variable
- = Independent Variable
- = Intercept
- = Slope of Regression Line

Example

Predicting House Price using only Median Income:

Characteristics

- Uses a single input feature.
- Produces a straight-line fit.
- Easy to understand and implement.
- Cannot capture complex nonlinear patterns.

Multiple Linear Regression

Multiple Linear Regression uses more than one independent variable to predict a dependent variable while maintaining a linear relationship.

Mathematical Model

Example

Characteristics

- Uses multiple input features.
- Captures the combined effect of several variables.
- Provides better prediction than Simple Linear Regression in many real-world applications.
- Still assumes a linear relationship between variables and target.

Polynomial Regression

Polynomial Regression is an extension of Linear Regression used when the relationship between variables is nonlinear. It introduces higher-order terms of the input feature.

Mathematical Model

Example

Characteristics

- Captures nonlinear relationships.
- Uses polynomial terms such as , , etc.
- Provides curved model fitting.
- Higher polynomial degrees may lead to overfitting.

Important Note

The term **Polynomial Regression** does not imply the use of multiple features. The word *Polynomial* refers to the presence of higher-order terms such as:

Even a single feature can be used to perform Polynomial Regression.

Comparison of Regression Techniques

Simple Linear Regression

- One feature
- Linear relationship
- Straight-line fit

Multiple Linear Regression

- Multiple features
- Linear relationship
- Better predictive ability

Polynomial Regression

- One or more features
- Nonlinear relationship
- Curved fitting

California Housing Dataset

The California Housing Dataset is a commonly used benchmark dataset for regression problems. It contains housing-related information collected from California census data.

Features

Feature	Description
MedInc	Median Income
HouseAge	Median House Age
AveRooms	Average Number of Rooms
AveBedrms	Average Number of Bedrooms
Population	Population in Block Group
AveOccup	Average Occupancy
Latitude	Latitude Coordinate
Longitude	Longitude Coordinate

Target Variable

MedHouseVal : Median House Value

In this experiment, the feature **Median Income (MedInc)** is selected because it has a strong correlation with house prices and allows clear visualization of polynomial fitting.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab / VS Code

Python Libraries

- NumPy
- Pandas
- Matplotlib
- Scikit-Learn

Algorithm

1. Import required libraries.

2. Load the California Housing Dataset.
3. Create a DataFrame from the dataset.
4. Select Median Income as the independent variable.
5. Select House Value as the target variable.
6. Split the dataset into training and testing sets.
7. Generate polynomial features of degree 2.
8. Train a Linear Regression model on polynomial features.
9. Predict house prices on testing data.
10. Calculate MSE, RMSE, and R^2 Score.
11. Visualize actual data and polynomial regression curve.
12. Analyze the obtained results.

Program

```
# Polynomial Regression on California Housing Dataset
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# Load dataset
housing = fetch_california_housing()
# Create DataFrame
df = pd.DataFrame(housing.data, columns=housing.feature_names)
# Target Variable
df['HouseValue'] = housing.target
# Independent and dependent variables
X = df[['MedInc']]
y = df['HouseValue']
```

```
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Polynomial Feature Transformation
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
# Model Training
model = LinearRegression()
model.fit(X_train_poly, y_train)
# Prediction
y_pred = model.predict(X_test_poly)
# Performance Evaluation
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("Mean Squared Error :", mse)
print("Root Mean Squared Error :", rmse)
print("R2 Score :", r2)
# Visualization
plt.figure(figsize=(8,5))
plt.scatter(
    X_test,
    y_test,
    color='blue',
    alpha=0.5,
    label='Actual Values'
)
sorted_indices = np.argsort(X_test.values.flatten())
plt.plot(
```

```
    X_test.values.flatten()[sorted_indices],
    y_pred[sorted_indices],
    color='red',
    linewidth=2,
    label='Polynomial Regression Curve'
)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Polynomial Regression on California Housing Dataset")
plt.legend()
plt.show()
```

Explanation of the Program

- The California Housing Dataset is loaded using Scikit-Learn.
- Median Income is selected as the predictor variable.
- PolynomialFeatures transforms the original feature into polynomial terms.
- A Linear Regression model is trained on the transformed features.
- Predictions are generated on the test dataset.
- Model performance is evaluated using MSE, RMSE, and R^2 Score.
- A polynomial curve is plotted to visualize the nonlinear relationship.

Sample Output

Mean Squared Error : 0.67

Root Mean Squared Error : 0.81

R2 Score : 0.49

Result Analysis

1. Polynomial Regression models nonlinear relationships effectively.
2. The model captures curved trends between Median Income and House Value.
3. Polynomial features improve flexibility compared to Simple Linear Regression.
4. The R^2 Score indicates how well the model explains variations in housing prices.
5. Excessively high polynomial degrees may lead to overfitting.

Result

Thus, Polynomial Regression was successfully implemented on the California Housing Dataset using Python, and housing prices were predicted by fitting a polynomial relationship between Median Income and House Value.

Conclusion

Thus, Polynomial Regression was successfully implemented on the California Housing Dataset using Python. The experiment demonstrated how polynomial features help capture nonlinear relationships between Median Income and House Value, resulting in a more flexible regression model compared to simple linear regression. The model's performance was evaluated using MSE, RMSE, and R^2 Score.

Viva Voce Questions

1. What is regression analysis?
2. Define Simple Linear Regression.
3. Define Multiple Linear Regression.
4. What is Polynomial Regression?
5. What is the difference between Linear Regression and Polynomial Regression?
6. Why is Median Income used in this experiment?
7. What is the purpose of PolynomialFeatures?
8. What is overfitting?
9. What is Mean Squared Error (MSE)?
10. What does the R^2 Score indicate?
11. Can Polynomial Regression be performed using a single feature?
12. What are interaction terms in polynomial feature engineering?
13. What is the significance of polynomial degree?
14. What is the role of train-test splitting?
15. Why is visualization easier with a single-feature polynomial model?